See that last line starting with rspec? That's where RSpec usefully gives you the exact command-line invocation you need to run just the failing spec. This is helpful. Later on, you'll see some other ways to get RSpec to rerun failing specs.

Our error is that you are calling project.done? and the done? method does not exist yet.

That's simple to clear, still in app/models/project.rb:

```
class Project
  def done?
  end
end
```

And when you do this and run rspec again, you finally get a more interesting error:

```
% rspec

F

Failures:

  1) Project considers a project with no tasks to be done
     Failure/Error: expect(project.done?).to be_truthy

       expected: truthy value
            got: nil
     # ./spec/models/project_spec.rb:6:in `block (2 levels) in <top (required)>'

Finished in 0.02281 seconds (files took 5.92 seconds to load)
1 example, 1 failure

Failed examples:

rspec ./spec/models/project_spec.rb:4
   # Project considers a project with no tasks to be done
```

You've now passed out of the realm of syntax and runtime errors and into the realm of assertion failures—the test runs, but the code does not behave as expected. You've expected that the value of project.done? will be truthy, which is to say any Ruby value that evaluates to true. But since your method doesn't return any value, you get nil.

Luckily, that has a simple fix:

basics/01/app/models/project.rb

```
class Project
  def done?
    true
  end
end
```